



Application Engineering Assignment

VIT Vellore'28 Batch

Duration 12 Hrs

Video Streaming Concurrency & Analytics Dashboard

Problem Overview

Imagine you are building the core analytics engine for a popular video streaming platform. The business team needs to answer one critical question: exactly how many people are actively watching a video at any given minute?

Your task is to build a full web application (Frontend and Backend) to calculate and visualize "Who is watching right now?". Simply having the app open does not equate to an active viewer. Users might pause the video, lose their network connection, or close the application abruptly. To track this behavior, the video player emits raw events (including play, pause, session start, session end, and periodic "heartbeats").

Your goal is to process this raw data to find the true number of active viewers. You will need to look at the patterns and time gaps in the events to come up with your own rules for when a user is 'active' versus 'inactive'. Your system must be flexible, it should be able to show the total active users overall, or let the business team select a specific time window to see exactly how many people were watching during that specific period. Finally, you will display these numbers on an interactive dashboard. Your system is required to answer total active users for the duration and minute level active users.

Requirements

Level 0: Data Pipeline & Active State Logic Resolution

Requirement: Ingest the provided telemetry data and apply the business rules to determine the exact "active" time intervals for every user session.

Details:

- **Data Source & Storage:** You will be provided with a `.csv` file containing simulated streaming telemetry events. Figure out a way to load them into your desired storage system.
- **Event Types:** The dataset contains `SESSION_START`, `PLAY`, `HEARTBEAT` (logged every 30s while active), `PAUSE`, and `SESSION_END` with timestamp.
- **Active Watch Rules:** Using these events, you must define the formula for when a user is actually watching. Think about edge cases: what happens if a user hits pause? What if they finish a session cleanly? What if the app crashes or the internet drops and the "heartbeats" suddenly stop?
- **Goal of this level:** Your primary goal is to establish your own definition of an "active viewer." By analyzing the provided dataset, you must design the logic that determines when a user is actively watching versus when they are not. Your definition could rely on a single specific event, a combination of events, or the time gaps between them. You must also account for edge cases (such as abrupt app closures or missing data). Once you have finalized your rules, use them to calculate the exact start and end timestamps for every active viewing period.

Note:

- There might not be a session end event always as users might have turned off the phone, application died, or anything else might have happened. You need to properly detect such things.
- Your design should at least be able to answer the basic questions like "total active users?", "active users between 10:30 to 11:00?", etc.

Level 1: The Concurrency API & High-Performance Architecture

Requirement: Build an API endpoint to calculate minute-by-minute concurrency overlaps, utilizing an architecture that avoids brute-force table scans on every request.

Details:

- **Core API:** Build an API endpoint that queries your processed data and returns minute-grain concurrency counts for any requested time range.
- **Tech Stack Flexibility & Performance:** You can independently choose the best tools, languages, and databases for the job. Aim to develop an approach that maximizes query performance. Select your technology stack carefully!
- **Architectural Options to Explore:** Devise an innovative and highly optimized algorithmic approach to efficiently handle this overlap calculation at scale.

Level 2: Executive Analytics Dashboard & UX Exploration

Requirement: Design and build an intuitive, flexible, and visually engaging web dashboard. Rather than just rendering basic charts, explore what views and user experiences best serve executive decision-makers.

Details:

- **UX & Dashboard Exploration:** We do not prescribe specific chart types or rigid layouts. Design a dashboard that provides maximum clarity into business requirement.
- **Multi-Dimensional Insights:** Enable users to inspect active viewership within any selected time window. Users should easily answer:
 - How many total viewers were active in a specific time frame?
 - What countries or device types are driving the current peak?
 - Which content/videos are currently drawing the largest active audience?
- **UX Best Practices:** Ensure the interface is responsive, handles loading and empty states gracefully, offers clear time-range controls, and allows seamless filtering across dimensions.

Entities

Event Entity

- **Event ID:** Unique identifier for the telemetry event.
- **Session ID:** Identifier binding all events in a single user's session.
- **User ID:** Unique identifier for the user.
- **Video ID:** Identifier of the video content being watched.
- **Timestamp:** The exact date and time of the event.
- **Event Type:** `SESSION_START`, `PLAY`, `HEARTBEAT`, `PAUSE`, or `SESSION_END`.
- **Device:** Device type (e.g., mobile, desktop, smart_tv).
- **Country:** ISO country code (e.g., IN, US, UK).

Video Entity (Optional / Contextual)

- **Video ID:** Unique identifier.
- **Title:** Title of the content.
- **Category:** Content genre/type (e.g., Live Sport, Entertainment, Movie).
- **Language:** Language ("IN", "US", "UK")

Sample Data Format

Two CSV files are provided as input:

1. events.csv - Raw telemetry events

Fields:

- event_id: UUIDv4 (e.g. "a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d")
- session_id: UUIDv4 (e.g. "f47ac10b-58cc-4372-a567-0e02b2c3d479")
- user_id: UUIDv4 (e.g. "7c9e6679-7425-40de-944b-e07fc1f90ae7")
- video_id: UUIDv4 (e.g. "550e8400-e29b-41d4-a716-446655440000")
- timestamp: UTC datetime (format: "YYYY-MM-DDTHH:mm:ss.SSSZ")
- event_type: one of `SESSION_START`, `PLAY`, `HEARTBEAT`, `PAUSE`, `SESSION_END`
- device: string (e.g. "mobile", "desktop", "smart_tv", "tablet")
- country: string (e.g. "IN", "US", "UK", "DE", "BR")

2. content.csv - Video dimension table

Fields:

- video_id: UUIDv4
- title: string (e.g. "Premier League: Arsenal vs Chelsea")
- category: string (e.g. "Movie", "Drama", "Live Sport", "News")
- language: string (e.g. "en", "hi", "de", "ja")

General Notes

- **Technology Flexibility:** Utilize any technology stack, library, or framework as we have no specific preferences.
- **Mobile App Development:** If you prefer developing a standalone mobile application instead of a web application, please consult with the Motorq team for approval.
- **Feature Implementation:** Focus on completing the Core Functionalities and System Design first, and then build out the UI.

Video Guidelines

Please include the following workflows in your video demonstration (showing your face is optional):

1. **Data Loading & Setup:** Briefly show your database and demonstrate how the dataset was ingested and stored.
2. **Core Logic & Edge Cases:** Show a specific user session from your database (for example, a session where the user paused, or where the heartbeat events suddenly stopped). Explain the rules you came up with to correctly separate their active watch time from their inactive time.
3. **High-Performance Architecture:** Explain your architectural choice (Delta, Bucket tables, Materialized views, etc.) and demonstrate your API query response latency.
4. **Dashboard UX & Drill-Downs:** Walk through your frontend dashboard. Demonstrate filtering and viewing concurrency trends across specific time windows, countries, devices, or content.

Guidelines for submission

- **How to submit?:** You will be provided with a link to a shared GitHub repository. You must submit your assignment by raising a Pull Request (PR) to this repository.
- **Directory Structure:** Inside the repository, create a new sub-directory named using the exact format `<name>-<reg-no>` (for example, `john-doe-24BCE1234`). All of your application files, frontend code, and backend code must be placed strictly inside this sub-directory.
- **Documentation:** Ensure that a `README.md` file is present in the root of your specific sub-directory. This file must contain clear, step-by-step instructions for any local setup required to run your application.
- **Important (Video Demo):** Record a working demonstration of your project following the Video Guidelines above. You must include the link to this video prominently in your `README.md` file. **Without a video demo, your project will not be reviewed.**